



Building RAG Agents with LLMs

Search Ranking: BM25, Embeddings & Evaluation

Jae Y. CHOI

NVIDIA DLI Certified Instructor & University Ambassador

본 자료의 저작권은 Jae Y. CHOI (jaeyeong2022@gmail.com)에게 있습니다.

본 자료는 자유롭게 이용할 수 있으나, 사용 시 저작자 및 출처를 반드시 표기해 주시기 바랍니다.

Copyright © Jae Y. CHOI (jaeyeong2022@gmail.com).

This material may be freely used, provided that the author and source are properly credited.

The Search Problem

Useful documents, near the top

Retrieve

Which documents might help?

6 documents become 5 candidates

Rank

Which candidate comes first?

A before B before C

Evaluate

Does this order help users?

Compare with relevance labels

Ranking score and evaluation metric have different roles.

One Query, One Teaching Corpus

Query: solar energy storage

Document	Tokens	Length
A	solar energy storage battery	4
B	solar battery storage	3
C	energy storage overview	3
D	solar system planet	3
E	energy market prices	3
F	storage furniture box	3

N = 6 documents Average length = 19 / 6 = 3.167 tokens



Lexical Retrieval

One-hot, Bag-of-Words, TF-IDF and BM25

One-hot Encoding

One token activates one vocabulary position

Token	solar	energy	storage	battery
solar	1	0	0	0
energy	0	1	0	0
storage	0	0	1	0

One-hot represents identity. It does not learn meaning.

Bag-of-Words

Counts combine token identities into a document vector

Text	solar	energy	storage	battery
solar solar storage	2	0	1	0
storage solar solar	2	0	1	0
solar energy battery	1	1	0	1

Same counts, same vector, even when word order changes.

Term Frequency

TF counts occurrences inside one document

solar solar solar

3

TF(solar)

Repeated mention

solar battery

1

TF(solar)

One mention

Frequency supplies evidence, but repetition can be misleading.

Document Frequency and IDF

A common term contributes less discrimination

$$\text{IDF}(t) = \log_2 \left(\frac{N}{\text{DF}(t)} \right)$$

N

Number of documents

DF(t)

Documents containing term t

log₂

Compresses the rarity ratio

Term	DF	IDF
solar	3	$\log_2(6/3) = 1.000$
storage	4	$\log_2(6/4) = 0.585$

DF counts documents. TF counts occurrences.

TF-IDF Weight

Local frequency multiplied by corpus-wide rarity

t, d

A term and a document

w(t,d)

Weight of term t in document d

TF × IDF

Frequent here, distinctive overall

$$w_{t,d} = \text{TF}(t, d) \times \text{IDF}(t)$$

Term in document A	TF	IDF	Weight
solar	1	1.000	1.000
energy	1	1.000	1.000
storage	1	0.585	0.585

TF-IDF gives term weights. A similarity rule gives the ranking.

Cosine Similarity

Compare vector direction after length normalization

$$\cos(q, d) = \frac{\mathbf{w}_q^T \mathbf{w}_d}{\|\mathbf{w}_q\|_2 \|\mathbf{w}_d\|_2}$$

w_q, w_d

Query and document weight vectors

Numerator

Multiply matching coordinates, then sum

Denominator

Product of full vector lengths

Step	Document A
1. Shared weighted terms	$1^2 + 1^2 + 0.585^2 = 2.342$
2. Full vector lengths	1.530×2.203
3. Divide	$2.342 / (1.530 \times 2.203) \approx 0.695$

0.695 is a similarity score, not a 69.5% relevance probability.

A TF-IDF Ranking

Computed on the same six-document corpus

Rank	Document	Cosine score	Matched query terms
1	A	0.695	solar, energy, storage
2	B	0.447	solar, storage
3	C	0.310	energy, storage
4=	D / E	0.172	solar / energy
6	F	0.060	storage

Compare ordering quality, not score scales across methods.

Why BM25?

A lexical score with controlled term repetition

Rarity

How common is the term?

IDF weighting

Frequency

How often does it appear here?

Diminishing extra gain

Length

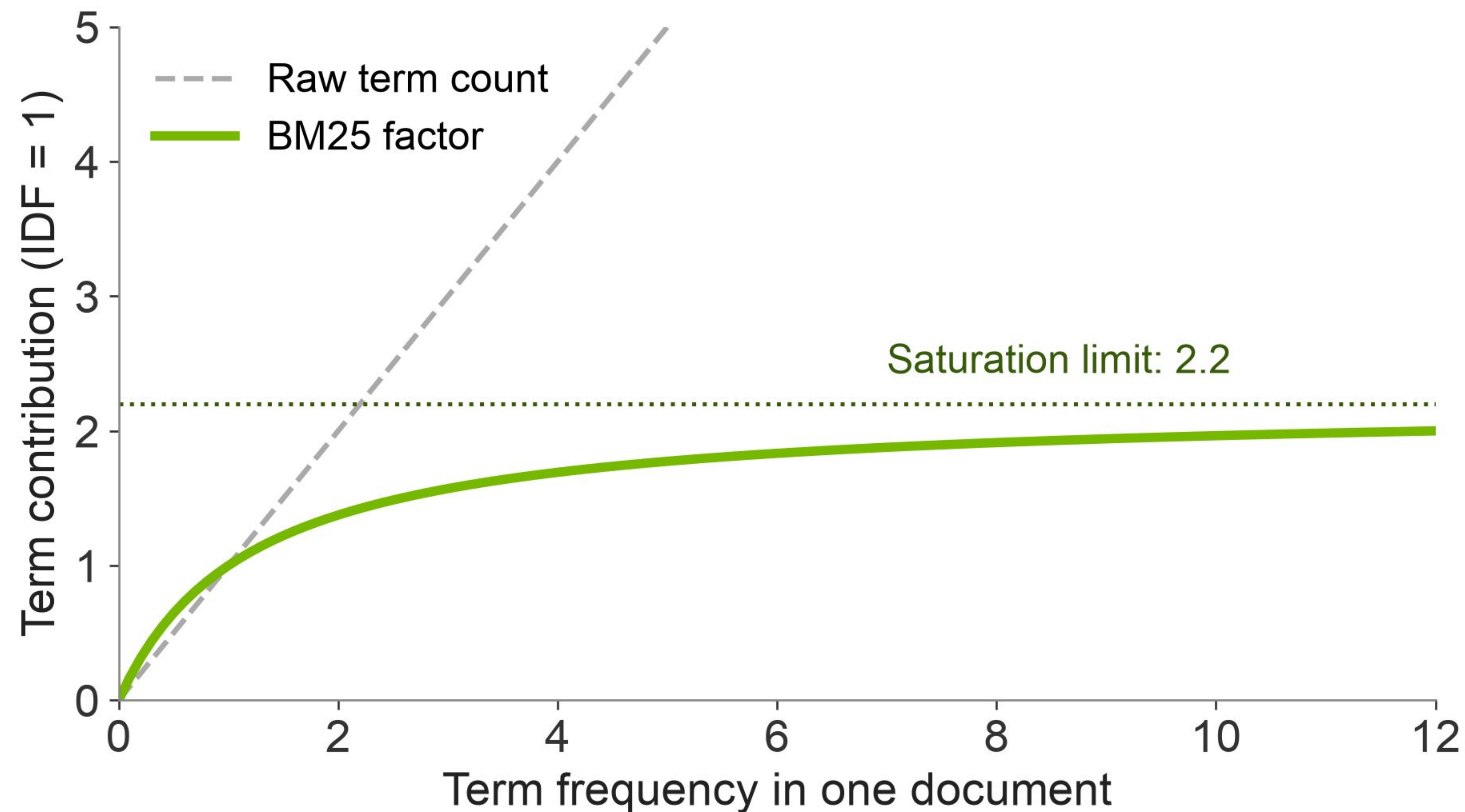
How long is this document?

Relative-length adjustment

BM25 balances rarity, repetition and document length.

BM25 Frequency Saturation

The first extra occurrence matters more than the eleventh



1 to 2

A noticeable increase

10 to 11

A much smaller increase

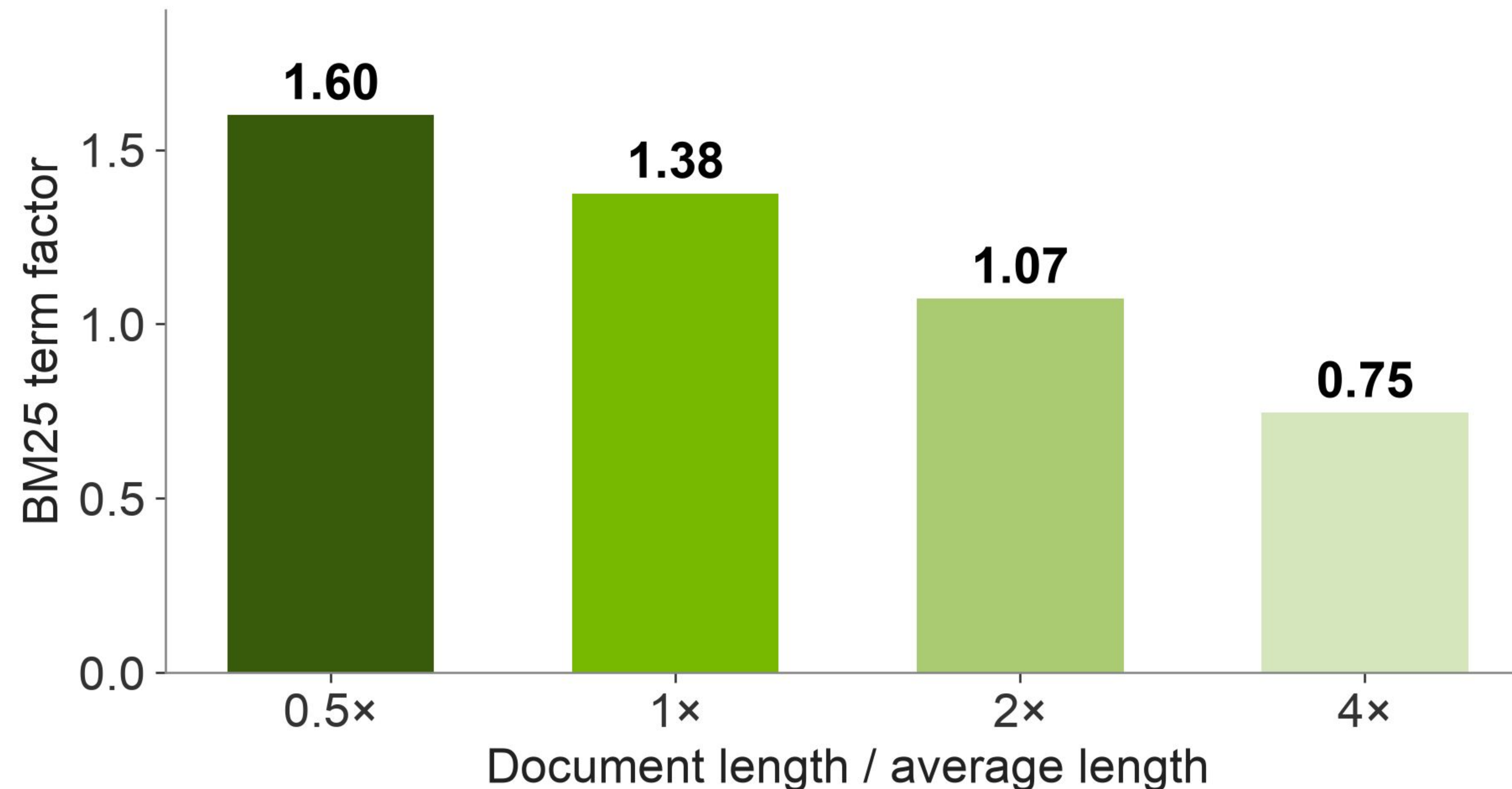
Fixed here

IDF = 1, length ratio = 1

k_1 controls the contribution of repeated terms.

BM25 Length Normalization

Two mentions carry different weight in different-length documents



Same TF

2 mentions

Short document

More concentrated evidence

Long document

Less concentrated evidence

b controls how strongly relative document length matters.

The BM25 Formula

One term contributes a weighted, normalized frequency

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1(1 - b + b|d|/\text{avgdl})}$$

Σ over t in q

Add the contribution of each distinct query term

IDF(t)

Rarity weight

Fraction

Frequency saturation with length normalization

Symbol	Meaning
$f(t,d), d $	Term count in d, document token count
avgdl	Average document length in the corpus
k_1, b	Frequency and length controls

Read it as: sum of rarity × adjusted frequency.

BM25 Worked Example

Document A, with $k_1 = 1.2$ and $b = 0.75$

N = 6

Corpus document count

ln

Natural logarithm for this BM25 variant

0.5 and +1

Smoothing in the positive IDF form

$$\text{IDF}(t) = \ln \left(1 + \frac{N - \text{DF}(t) + 0.5}{\text{DF}(t) + 0.5} \right)$$

Term in A	BM25 IDF	Frequency factor	Contribution
solar	0.693	0.903	0.626
energy	0.693	0.903	0.626
storage	0.442	0.903	0.399

Full-precision sum: $\text{BM25}(q, A) \approx 1.650$

Lexical Retrieval Trade-offs

Exact wording remains valuable

Works well for	Can fail when	Useful response
Names, IDs, error codes	Users paraphrase the topic	Add semantic candidates
Fast, inspectable matching	Tokens split poorly	Improve tokenization
No neural-model training	Word overlap hides wrong intent	Evaluate or rerank candidates



Neural Retrieval

Encoders, embeddings and candidate reranking

Static and Contextual Embeddings

A word can change meaning with its context

river bank

Land beside a river

bank account

A financial institution

Representation	Input	What changes?
One-hot	A token identity	Only its vocabulary position
Word2Vec / GloVe	A word type	Learned static vector
Contextual encoder	Tokens in a sentence	Token vectors depend on context

Contextual token vectors still need a suitable retrieval representation.

Encoder and Pooling

Token representations become a query or document embedding

01 Tokenize

solar / energy / storage

Convert text into model tokens

02 Encode

One hidden vector per token

Use context to represent tokens

03 Pool

Mean, CLS or last token

Produce one text vector

04 Compare

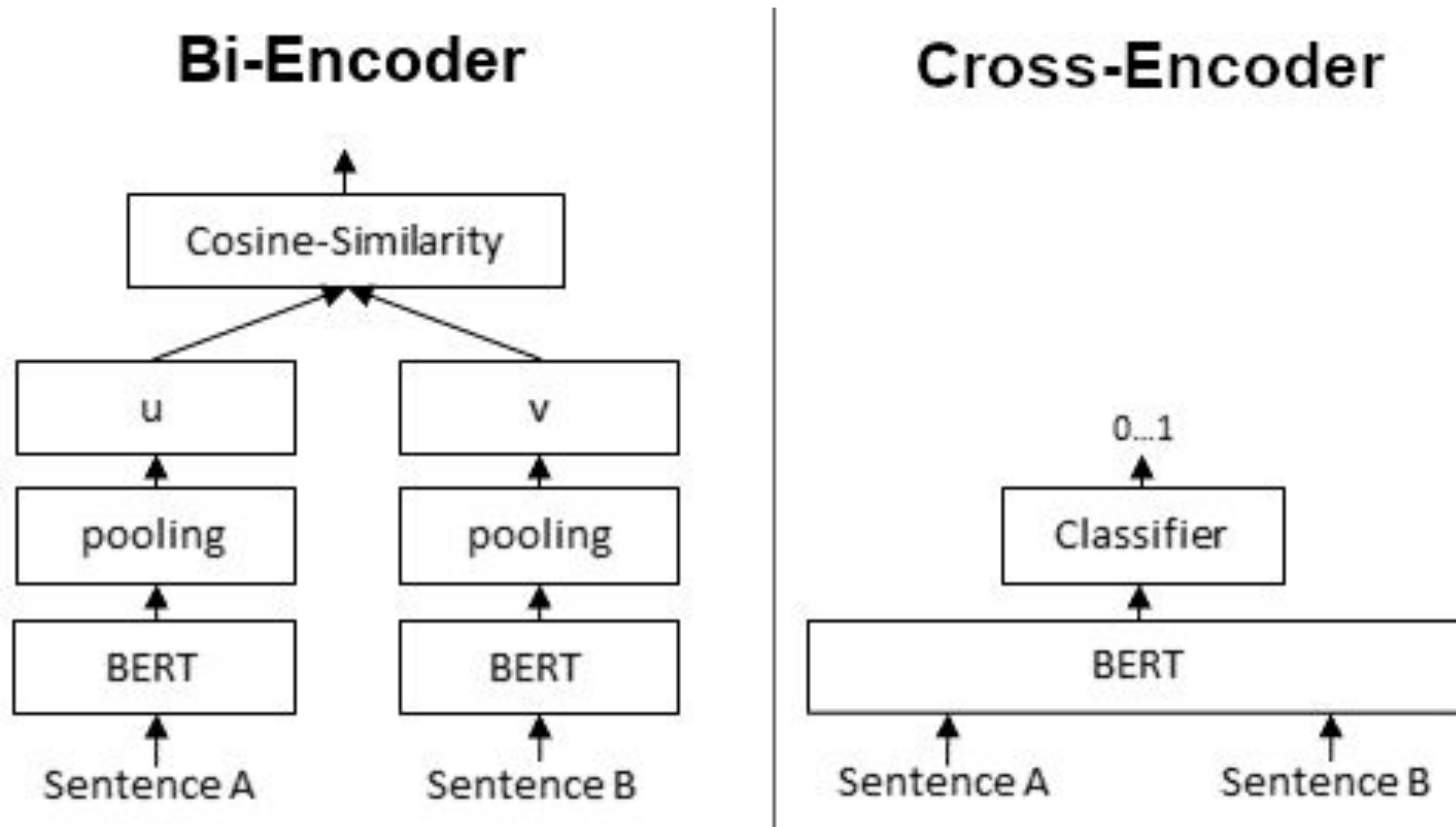
Cosine or dot product

Score query–document similarity

Use the model's documented pooling and query format.

Bi-Encoder vs. Cross-Encoder

Independent vectors or a joint relevance judgment



Precomputed document vectors make broad retrieval practical.

Bi-Encoder Retrieval

Documents are encoded before the query arrives

When	Work	Reusable output
Indexing	Encode every document chunk	Stored document vectors
Query time	Encode the user query	One query vector
Search	Nearest-neighbor lookup	Top candidate IDs

The vector index searches embeddings. It does not create their meaning.

Learning Retrieval Similarity

Relevant examples should score above confusing negatives

Query: solar energy storage

Relevant pair

photovoltaic battery
systems

Different words, same need

Hard negative

solar system planets

Shared words, wrong topic

Unrelated pair

living-room storage
boxes

Wrong meaning of storage

Embedding quality depends on the relevance relationships used in training.

Dense Retrieval Trade-offs

Semantic similarity can help and mislead

Advantage	Failure to inspect	Example
Paraphrase matching	Wrong topic with similar wording	solar power vs. solar system
Cross-lingual potential	Language or domain gaps	Korean technical queries
Fast vector lookup	Exact identifiers get blurred	Part number XJ-2047

Qwen3 Embedding and Reranker

One family, two different output types

Embedding

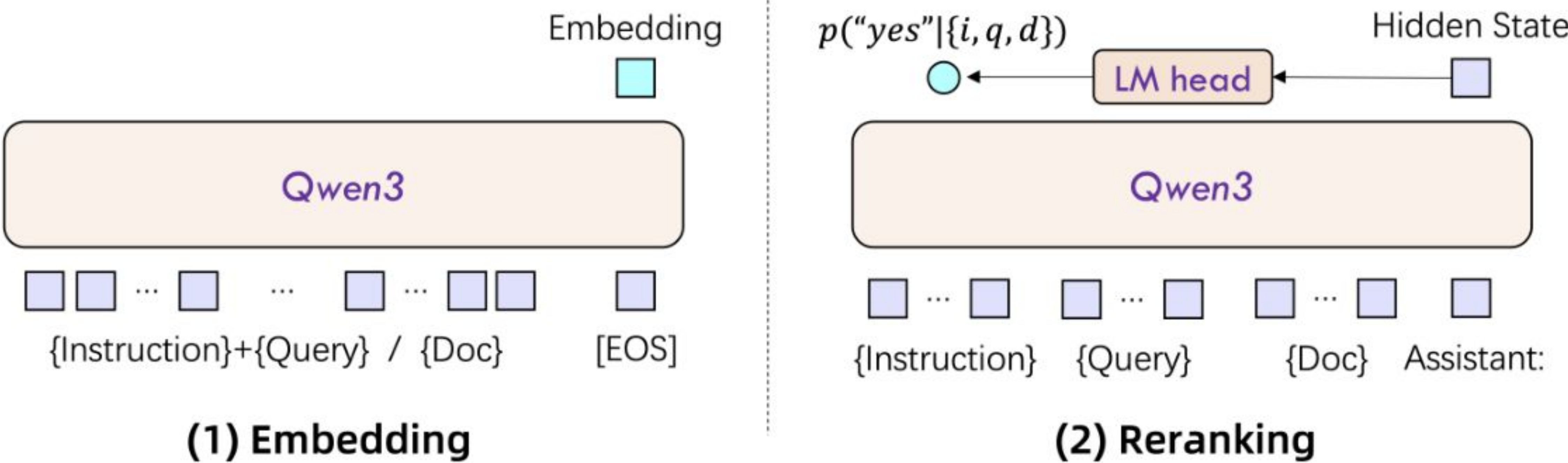
A reusable vector

Reranker

A query–document score

Sizes

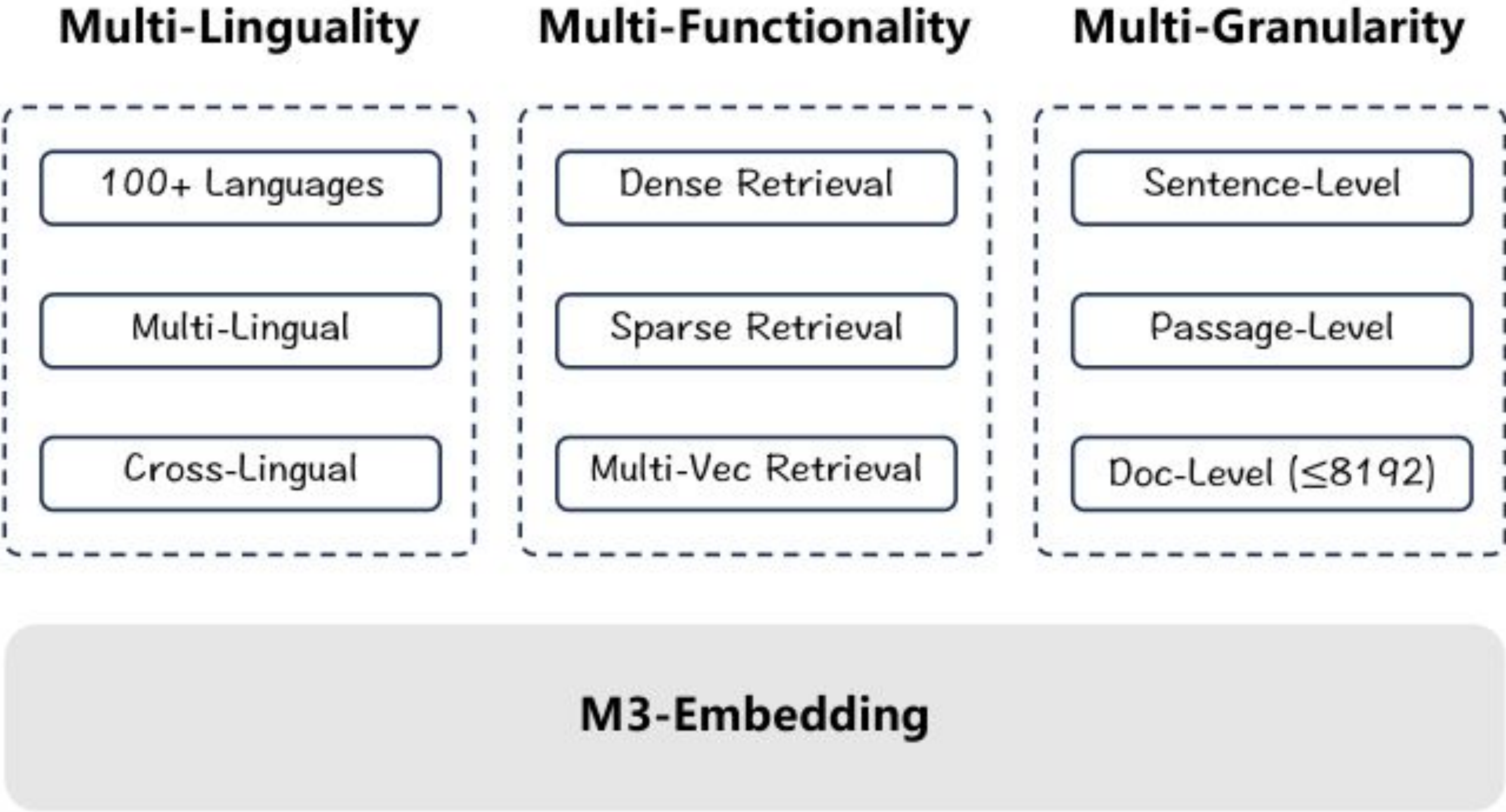
0.6B, 4B and 8B



Embedding model for indexing. Reranker model for candidate scoring.

BGE-M3

Dense, sparse and multi-vector retrieval in one model



Dense

One 1024-dimensional vector

Sparse

Vocabulary-based weights

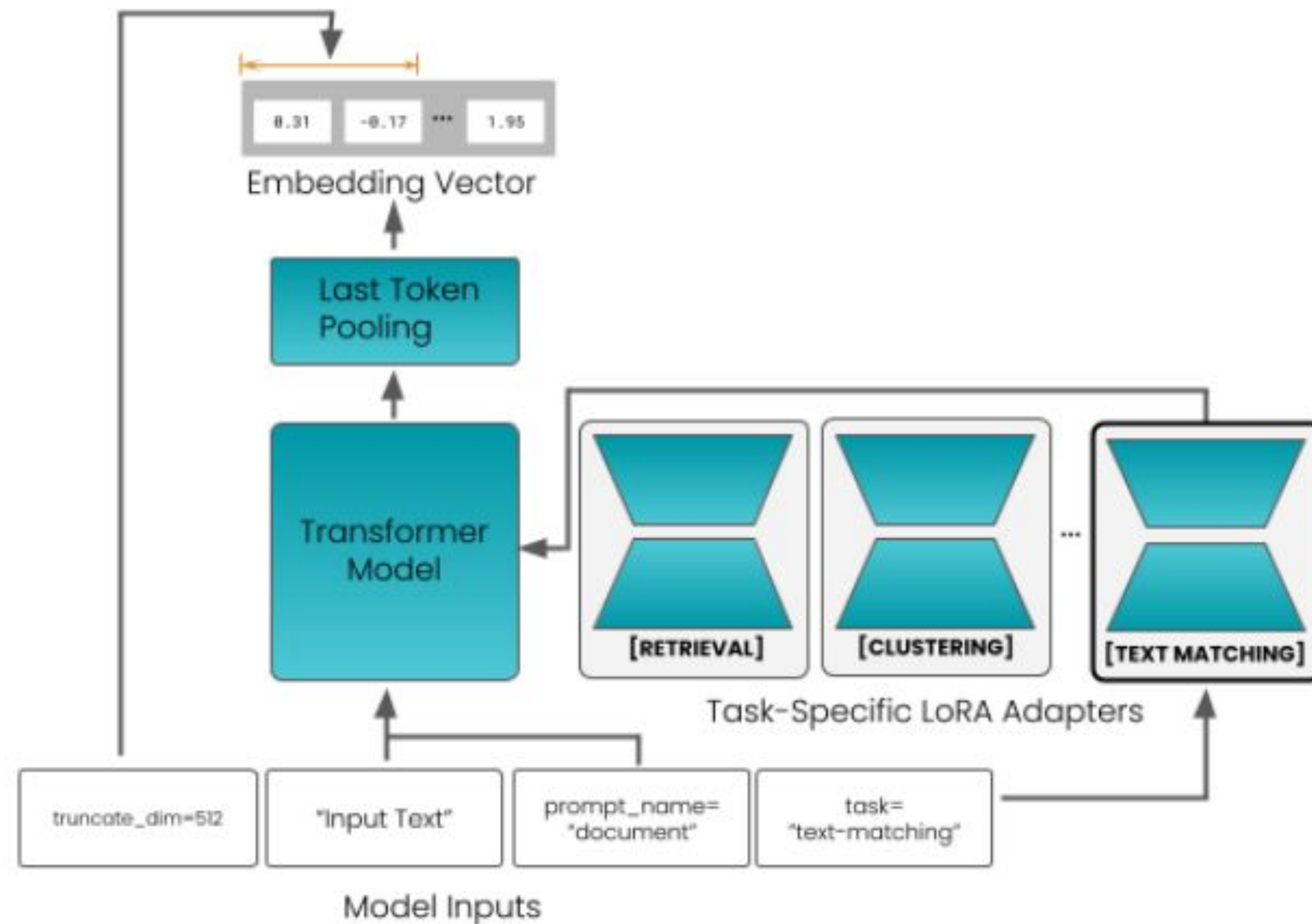
Multi-vector

Token-level interaction

The selected output mode determines the retrieval method.

Jina Embeddings v5 Text

A 2026 example of task-specific embedding design



Task adaptation

Choose the intended task

Pooling

One output vector

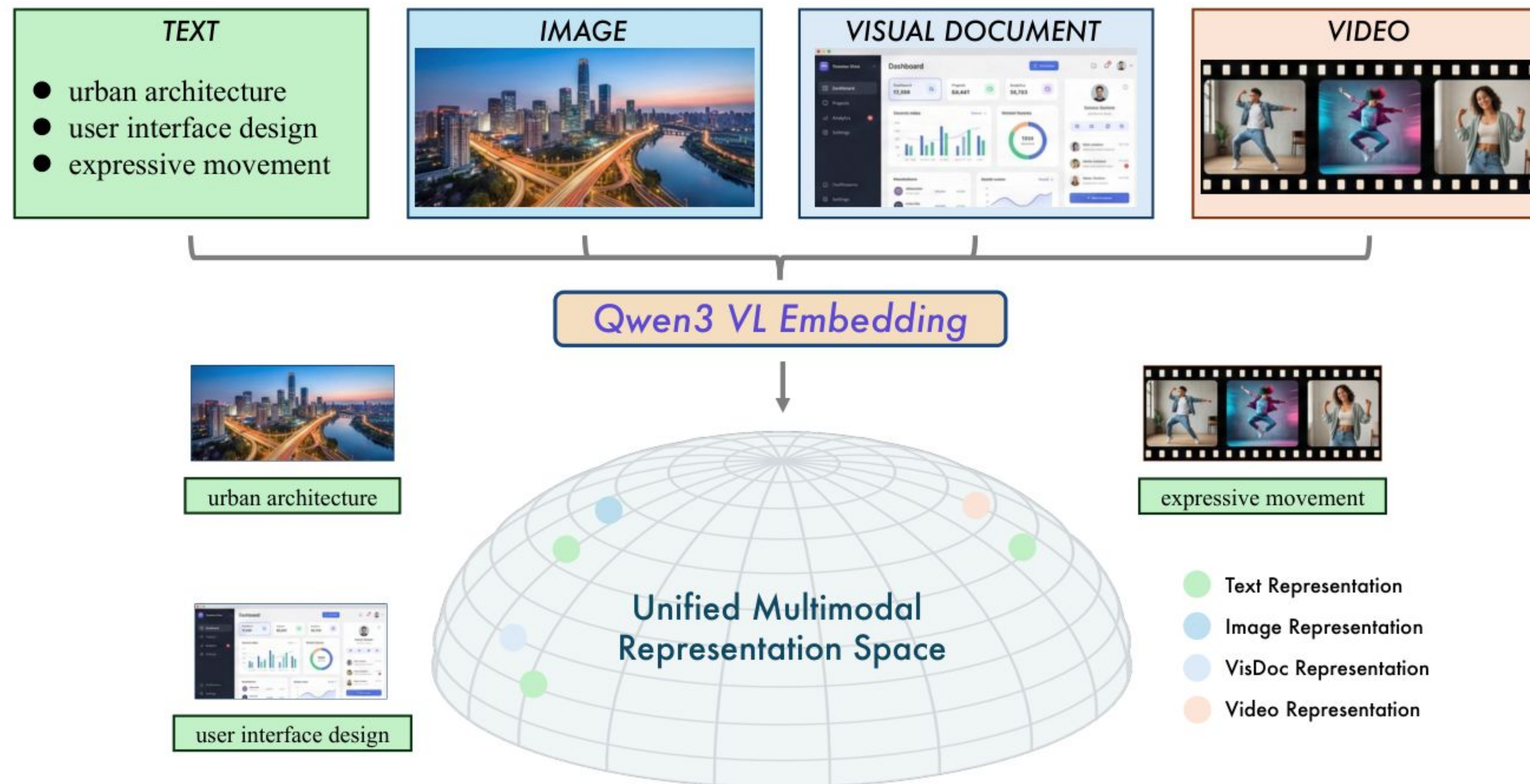
Variants

Small and nano

Task configuration can matter as much as model size.

Multimodal Retrieval

Qwen3-VL can represent text, images, pages and video



The document's modality changes the appropriate model and index.

Recent Model Options

Representative families, not a universal leaderboard

Family	Main use	What to check
Qwen3 Embedding / Reranker	Text retrieval and reranking	Instruction, size, latency
BGE-M3	Multilingual hybrid representation	Dense / sparse / multi-vector mode
Jina v5-text / v5-omni	Task-aware text / multimodal search	Task and modality configuration
Qwen3-VL Embedding / Reranker	Visual and multimodal retrieval	Page and video handling
Voyage 4 family	Managed or local text embeddings	Serving mode, shared-space support

Benchmark fit, language and operating cost decide the shortlist.

Cross-Encoder Scoring

Each candidate is read together with the query

$$s(q, d) = g_{\theta}([q; d])$$

$[q; d]$

The query and candidate in one input

g_{θ}

A trained joint scoring model

$s(q, d)$

A relevance score for this pair

Model input	Output	Reuse across different queries?
query + document A	score for A	Score must be recomputed
query + document B	score for B	Score must be recomputed

Joint scoring trades more computation for finer query–document interaction.

Reranker Families

Reranker names describe a role, not one architecture

Example	Scoring approach	Use and limitation
BGE-reranker-v2-m3	Joint query–document scoring	Multilingual candidate refinement
Qwen3-Reranker	Instruction-aware yes/no scoring	Flexible, but more compute at larger sizes
Cohere Rerank 4	Managed reranking API	Operational simplicity, API dependency
Listwise LLM reranking	Judge several candidates together	More context and ordering complexity

Match the reranker to language, input length and latency budget.

The Candidate Ceiling

A reranker can only reorder the documents it receives

Corpus: A · B · C · D · E · F

Useful document

A

RETRIEVED CANDIDATES

B · C · D

AFTER RERANKING

Still only B · C · D

A was never passed to the reranker.

Candidate recall limits downstream ranking quality.

Learned Sparse Retrieval

SPLADE learns vocabulary weights and expansion

Method	Coordinates	Weights
Bag-of-Words	Vocabulary terms	Observed counts
TF-IDF / BM25	Vocabulary terms	Frequency and rarity
SPLADE	Vocabulary terms	Learned importance and expansion

Sparse representation can also be learned and semantic.

Late Interaction

ColBERT keeps multiple vectors per document

$$s(q, d) = \sum_{i=1}^{|q|} \max_{j \in d} \mathbf{q}_i^T \mathbf{d}_j$$

$\mathbf{q}_i, \mathbf{d}_j$

Query-token and document-token vectors

max over j

Best document-token match for each query token

Σ over i

Add those best matches

Single-vector

One vector summarizes a document

Smaller index

Multi-vector

Several vectors retain token-level evidence

More storage and interaction work

Late interaction retains token detail while reusing document representations.

Hybrid Search with RRF

Combine ranks without pretending raw score scales match

$$\text{RRF}(d) = \sum_{s \in S} \frac{1}{c + \text{rank}_s(d)}$$

S

The set of ranked result lists

rank_s(d)

Document position in list s, starting at 1

c

A smoothing constant, 60 in this example

Document	Lexical rank	Dense rank	RRF
A	1	3	0.03227
B	2	1	0.03252
C	3	2	0.03200

B scores highest because it ranks near the top in both lists.

Retriever and Reranker Trade-offs

Each method retains different information

Method	Strength	Cost or blind spot
BM25	Exact terms, efficient baseline	Vocabulary mismatch
Dense bi-encoder	Paraphrases, reusable vectors	Exact detail can blur
Learned sparse	Learned term matching	Sparse index and model cost
Late interaction	Token-level matching detail	More vectors to store
Cross-encoder reranker	Joint relevance scoring	Cost per candidate, candidate ceiling

A useful pipeline balances coverage, ordering quality and operating cost.

Learning to Rank

Learn which documents should appear first

Pointwise

A document grade

A has relevance grade 3

Pairwise

Which document is better?

A should rank above B

Listwise

Quality of the whole order

A, B, C should outrank C, B, A

Training objective and evaluation metric need not be identical.

A Search Pipeline to Evaluate

Candidate generation and final ranking need different checks

Stage	Example choice	Main check
Prepare documents	Chunking and metadata filters	Evidence stays intact
Generate candidates	BM25 + dense retrieval	Recall@candidate K
Combine	RRF and deduplication	Coverage and overlap
Rerank	Cross-encoder on candidates	NDCG@display K
Serve	Top results with document links	Latency and real usefulness

Choose metrics for the stage being evaluated.



Ranking Metrics

Coverage, early success and graded usefulness

Relevance Judgments

Define useful before calculating a metric

Document	Teaching grade	Binary relevant?
A: direct help	3	1
B: useful help	2	1
C: partial help	1	1
D / E / F: not useful	0	0

R = 3 relevant documents in the judged corpus.

The Ranking Used Below

A deliberately imperfect order for metric calculations

Rank	1	2	3	4	5	6
Document	D	A	E	B	C	F
Grade	0	3	0	2	1	0

Relevant documents appear at ranks 2, 4 and 5.

Precision@K

Among the shown results, how many are useful?

$$P@K = \frac{\sum_{i=1}^K \text{rel}_i}{K}$$

K

How many top results we inspect

rel_i

1 if rank i is relevant, otherwise 0

Numerator

Relevant results inside the top K

Rank	1	2	3	4	5
Document	D	A	E	B	C
Relevant	0	1	0	1	1

$$P@5 = 3 / 5 = 0.60$$

Strength: top-result purity. Blind spot: missed evidence and internal order.

Recall@K

Of all useful documents, how many did we retrieve?

$$\text{Recall@K} = \frac{\sum_{i=1}^K \text{rel}_i}{R}$$

R

All relevant documents in the judgment set

Numerator

The same count used in Precision@K

Denominator

All relevant documents, not all returned documents

Rank	1	2	3	4	5
Document	D	A	E	B	C
Relevant	0	1	0	1	1

$$\text{Recall@5} = 3 / 3 = 1.00$$

Strength: coverage. Blind spot: noise and ordering within top K.

Same Set, Different Reading Experience

Precision and Recall ignore order within top K

Original order

Rank	1	2	3	4	5
Document	D	A	E	B	C
Grade	0	3	0	2	1

Improved order

Rank	1	2	3	4	5
Document	A	B	C	D	E
Grade	3	2	1	0	0

Both lists: $P@5 = 0.60$ and $Recall@5 = 1.00$

Hit@K and Reciprocal Rank

One useful result, and how soon it appears

$$RR(q) = \frac{1}{r_{\text{first}}}$$

r_{first}

Position of the first relevant result

No hit

RR is 0 if no relevant result is returned

Metric	Question	Same-list example
Hit@3	Any relevant result in top 3?	Yes: 1
RR	Where is the first relevant result?	Rank 2: $1/2 = 0.50$

Hit and RR focus on first success, not complete evidence coverage.

Mean Reciprocal Rank

Average the first-success score across queries

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \text{RR}(q)$$

Q, |Q|

Evaluation queries and their count

RR(q)

First-relevant-position score for one query

Mean

Each query receives equal weight

Query	First relevant rank	RR
Query 1	2	0.50
Query 2	1	1.00

$$\text{MRR} = (0.50 + 1.00) / 2 = 0.75$$

Average Precision

Read Precision only where a relevant result appears

$$AP = \frac{1}{R} \sum_{i=1}^L P@i \text{ rel}_i$$

L

Length of the evaluated result list

rel_i

Include only relevant positions

R

Total relevant documents, including any missed ones

Relevant rank	Precision at that rank	Contribution
2: document A	1 / 2	0.50
4: document B	2 / 4	0.50
5: document C	3 / 5	0.60

$$AP = (0.50 + 0.50 + 0.60) / 3 \approx 0.533$$

Mean Average Precision

First average within a query, then average across queries

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

AP(q)

Average Precision for one query

|Q|

Number of evaluation queries

Equal weight

Each query contributes one AP value

Query	Relevant ranks / total relevant	AP
Query 1	2, 4, 5 / R = 3	0.533
Query 2	1, 3 / R = 2	0.833

$$\text{MAP} \approx (0.533 + 0.833) / 2 \approx 0.683$$

Binary and Graded Relevance

Some useful results are more useful than others

Grades 1, 2, 3

Rank	1	2	3
Document	C	B	A
Grade	1	2	3

Grades 3, 2, 1

Rank	1	2	3
Document	A	B	C
Grade	3	2	1

Binary metrics tie. Graded relevance distinguishes the two orders.

Discounted Cumulative Gain

Useful results contribute more when they appear earlier

$$\text{DCG@K} = \sum_{i=1}^K \frac{2^{g_i} - 1}{\log_2(i + 1)}$$

g_i

Relevance grade at rank i

$2^{g_i} - 1$

Gain from that grade

$\log_2(i + 1)$

Rank-based discount in the denominator

Part	Read as	Example
Gain	How useful is it?	Grade 3 gives 7
Discount	How early is it?	Rank 2 divides by $\log_2 3$
Sum	Total discounted usefulness	Add top-K contributions

DCG combines grade, position and cumulative contribution.

DCG Worked Example

The original order D, A, E, B, C

Rank	Document / grade	Gain	Discounted contribution
1	D / 0	0	0
2	A / 3	7	$7 / \log_2 3 = 4.417$
3	E / 0	0	0
4	B / 2	3	$3 / \log_2 5 = 1.292$
5	C / 1	1	$1 / \log_2 6 = 0.387$

DCG@5 = 6.095 using full precision before rounding.

Normalized DCG

Compare actual gain with the best possible ordering

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

DCG

Discounted gain of the actual ranking

IDCG

DCG of the ideal order for the same judgments

Ratio

Share of ideal discounted gain

Ordering	Grades	DCG@5
Actual: D A E B C	0, 3, 0, 2, 1	6.095
Ideal: A B C D E	3, 2, 1, 0, 0	9.393

$$\text{NDCG@5} = 6.095 / 9.393 \approx 0.649$$

What Changes After Reranking?

Same top-5 set, improved positions

Metric	D A E B C	A B C D E	What it notices
P@5	0.60	0.60	Useful count
Recall@5	1.00	1.00	Coverage
RR	0.50	1.00	First useful rank
AP	0.533	1.000	All relevant positions
NDCG@5	0.649	1.000	Grades and positions

The metric should match the improvement you want to measure.

Metric Strengths and Blind Spots

A score answers one particular question

Metric	Useful for	Does not fully describe
Precision@K	Top-result purity	Missed documents and order
Recall@K	Evidence coverage	Noise and order
Hit@K / MRR	First useful answer	All evidence and graded utility
MAP	Ordering multiple relevant results	Graded usefulness
NDCG@K	Graded usefulness near the top	Latency, cost or factual answer quality

Metric selection starts with the intended search experience.

A Fair Model Comparison

The same evaluation problem for every candidate

Hold fixed

Why it matters

Query set, corpus version and judgments

Avoid comparing different tasks

Chunking, filters and cutoff K

Candidate opportunity must be comparable

Gain, tie and empty-query rules

Metric definitions affect the result

Validation vs. held-out queries

Do not tune on the final test

Latency, index size and cost

Quality has operational trade-offs

Model leaderboards suggest candidates. Local evaluation selects a system.

Diagnosing a Search Failure

Change the stage that lost the useful evidence

Observation	Inspect first	Helpful measurement
Useful document never appears	Chunking, filters, retriever	Candidate Recall@K
Useful candidate ranks too low	Reranker or score fusion	NDCG@K / MRR
Relevant evidence, wrong answer	Generator and evidence use	Answer correctness / faithfulness
Good quality, slow responses	Candidate count and serving	Latency, throughput and cost

Good retrieval supports good answers, but does not guarantee them.

Check Your Understanding

Use the first three ranks. Then think about missing candidates.

0 1 1 0

Ranks 1–4 · Total relevant documents: $R = 3$

Precision@3?

Relevant in top 3 / shown positions

Recall@3?

Relevant in top 3 / all relevant

Reciprocal Rank?

First useful position

Missing candidate?

Think about candidate membership

Count first. Choose the denominator second.

Worked Answers

The denominator tells you which question you answered

Metric or question	Answer	Reason
Precision@3	2 / 3	Two useful results in three slots
Recall@3	2 / 3	Two of three relevant documents found
RR	1 / 2	First useful result is at rank 2
Missing candidate	Not by reordering	Reordering does not add documents

A metric value is meaningful only together with its definition.

Choosing a Starting System

An experiment plan, not a universal prescription

Need	Reasonable baseline	Next comparison
Exact names and identifiers	BM25 with careful tokenization	Dense or sparse expansion
Paraphrase-heavy text	A suitable bi-encoder	Hybrid candidate coverage
Good candidates, weak ordering	A reranker on the candidate set	NDCG and latency
Visual pages or mixed media	A modality-compatible retriever	OCR / text baseline and page retrieval

Start with a baseline and a judged query set. Add complexity when measured.

Key Distinctions

Representation, scoring, ranking and evaluation

Concept	What to remember
One-hot / Bag-of-Words	How tokens and documents are represented
TF-IDF / BM25	How lexical evidence receives weight
Bi-encoder / Cross-encoder	Independent representation or joint scoring
Reranker / Hybrid search	Refine candidates or combine signals
Ranking metrics	Measure the user outcome you care about

Useful evidence must be found, ordered and evaluated.



Appendix

Implementation choices and metric edge cases

Scoring Conventions

A reproducible result needs a precise definition

Item	Convention used here
Tokenization	Lowercase, whitespace, no stemming
TF-IDF	Raw TF $\times \log_2(N / DF)$
Cosine	Full-vocabulary L2 norms
BM25	Positive smoothed In IDF, $k_1=1.2$, $b=0.75$
Repeated query terms	One contribution per distinct query term
Numbers	Calculate first, round only for display

Formula variants are part of the experimental setup.

Metric Edge Cases

Document the policy before comparing scores

Case	Policy to state
AP@K	R or $\min(R, K)$ denominator
Fewer than K results	Fixed K or returned count for Precision
R = 0 or IDCG = 0	Exclude or assign zero, report query count
Ties and duplicate documents	Deterministic tie rule, unique document IDs
Unjudged results	Keep judgment gaps visible
Multiple relevant chunks	Define document-level or chunk-level relevance

A metric name alone is not a complete evaluation specification.



Building RAG Agents with LLMs

Search Ranking: BM25, Embeddings & Evaluation

Jae Y. CHOI

NVIDIA DLI Certified Instructor & University Ambassador

본 자료의 저작권은 Jae Y. CHOI (jaeyeong2022@gmail.com)에게 있습니다.

본 자료는 자유롭게 이용할 수 있으나, 사용 시 저작자 및 출처를 반드시 표기해 주시기 바랍니다.

Copyright © Jae Y. CHOI (jaeyeong2022@gmail.com).

This material may be freely used, provided that the author and source are properly credited.